Webcam Hand Gestures for Desktop Interaction

Jaden Shelton
Colorado State University
Fort Collins, Colorado, USA
jaden.shelton@colostate.edu

Abstract

To Do

CCS Concepts

• **Human-centered computing → Gestural input**; *User studies*; Desktop GUIs.

Keywords

hand gesture recognition, webcam, desktop interaction, MediaPipe, multimodal input, 3D object manipulation, HCI

ACM Reference Format:
Jaden Shelton. 2026. Webcam Hand Gestures for Desktop Interaction. In *Proceedings of CS 465: Human-Computer Interaction (CS 465).* ACM, New York, NY, USA, 3 pages. https://doi.org/XXXXXXX.XXXXXXX

1 Introduction

Keyboard and mouse have been the dominant input devices for desktop and laptop computing for decades, and despite how much computers have advanced and changed over the years, that has remained the same. Augmented and virtual reality systems have pushed hand gesture interaction forward in HCI research, and these computing environments offer more direct and immersive ways to manipulate digital content. Most of this innovation has been confined to the specialized hardware that AR/VR uses, but open-source solutions like MediaPipe have matured and could bring similar interaction techniques to the standard desktop or laptop environment.

This brings up the question: can a small set of hand gestures be used alongside keyboard and mouse to meaningfully improve HCI with the types of computers we are accustomed to using every day? Particularly, could gesture input benefit the experience of performing 3D object manipulation tasks on a normal desktop or laptop? Prior work on hand gesture recognition has largely focused on recognition accuracy, sensing technology comparisons, and gesture design in specialized environments [2, 5]. Studies that do examine gesture usability for desktop tasks, such as Farhadi-Niaki et al., have found promise in finger-based gestures for object manipulation but have typically relied on dedicated depth cameras rather than the average webcam hardware most users already own [1]. In addition, research on gesture fatigue has made it clear that gesture interaction works best as a complement to existing input devices rather than a wholesale replacement. It was found that users could not sustain mid-air gesture input for very long without significant physical discomfort [3].

This project investigates whether webcam-based hand gesture input, designed as a supplement to mouse and keyboard input, could improve usability and task performance for 3D object manipulation on a standard desktop or laptop computer.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
CS 465, Colorado State University, Fort Collins, CO
© 2026 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN N/A
https://doi.org/XXXXXXX.XXXXXXX

2 Related Work

Research on hand gesture recognition (HGR) for human–computer interaction has been ongoing for decades, covering sensing technologies, gesture design, physical ergonomics, and multimodal interaction. This section reviews the prior work most relevant to this project: HGR sensing approaches, gesture design for intuitive interaction, the overall usability of gesture-only input, and multimodal gesture approaches in interactive environments.

2.1 Hand Gesture Recognition Technologies

There is a broad range of HGR research that covers many different sensing approaches, each with its own advantages and disadvantages. Guo et al. reviewed several HGR technologies utilizing gloves, cameras, ultrasound, and electromyography techniques to detect and identify hand gestures, and praised camera-based recognition for having a relatively high accuracy while also being more enjoyable for users since it did not require any special hardware [2]. This allows for a more comfortable experience that is well suited for dynamic gesture recognition, though one is limited by the camera's field of view.

Rautaray and Agrawal explored camera-based HGR systems thoroughly and identified three fundamental phases common to all systems: detection, tracking, and recognition [5]. Their survey found that desktop applications were one of the most frequently targeted areas in HGR research, even though the technology has not yet found many real-world use cases. They identified several obstacles to gesture interaction: reliability, speed, and intuitiveness [5].

The technology this project uses, MediaPipe, was developed by Zhang et al. at Google Research [8]. The hand recognition in MediaPipe is a two-stage pipeline made up of the BlazePalm detector, which localizes the hand in the full frame, and a landmark model that predicts 21 2.5D key points on the detected hand. The pipeline is optimized for real-time inference on devices such as a desktop or laptop and does not require any specialized hardware. This design makes MediaPipe well suited for testing the usability of gesture input in a common everyday desktop or laptop environment.

2.2 Practical Gesture Design

A major challenge in gesture-based HCI is designing gestures that feel intuitive to users. Much prior research has had gestures defined from the system designer's perspective. In a paper by Wobbrock et al., it is argued that choosing gestures should be based on actual user behavior [7]. In their study, Wobbrock et al. recruited 20 non-technical participants and asked them to invent gestures for given commands, producing a user-defined gesture set based on the consensus of participant preferences. Overall, they found that even in non-desktop environments, participants still associated certain actions with common Windows or Mac interface elements. They also found that some commands did not generate a decisive gesture, suggesting that hybrid interfaces and multimodal interaction could be more advantageous in those scenarios [7].

Another study by Farhadi-Niaki et al. tested a vision-based gesture recognition system in a simulated desktop environment [1]. Their system tested both finger-only gestures and full-arm gestures for common desktop tasks, and they evaluated several usability factors including efficiency, precision, ease of use, naturalness, and user satisfaction. They found that finger-based inputs were superior to arm-based ones in the long run, largely because arm gestures caused significantly more fatigue and felt less natural to users [1]. This is a primary reason why this project focuses on finger and hand-based gestures rather than large arm-based movements, creating a more compact interaction technique for a seated desk-based workflow.

2.3 Gesture Limitations

Even with smaller hand and finger-based gestures, this input type as a primary modality can cause user fatigue. Hansberger et al. studied this problem directly and compared mid-air gestures, keyboard input, and supported gestures over a 30-minute interaction session [3]. Their results were significant: only 25% of participants could complete a full 30-minute session using conventional mid-air gestures, compared to 100% completion for both the keyboard and supported gesture conditions. They referred to the arm fatigue caused by mid-air gestures as *gorilla arm syndrome*, a significant limiting factor when designing HGR interaction systems. These findings support the multimodal design for this project. Rather than replacing keyboard and mouse entirely, hand gestures serve as a supplement and can be performed from a supported, more relaxed position when seated at a desk to reduce potential fatigue.

2.4 Object Manipulation and 3D Environments

Gesture interaction has been studied extensively in 3D and immersive environments, often combined with different modalities to facilitate object manipulation. Rodriguez et al. conducted a study with artists using a VR 3D sketching application and examined which interaction modalities the artists preferred for different tasks [6]. The pattern they recognized was that artists particularly favored gestures for direct object manipulation tasks such as sculpting and erasing, but preferred their controller for menu navigation and tool selection. This shows the alignment between hand gestures and more spatial, physical operations—an alignment that is consistent with the gesture command set chosen for this project.

Plabst et al. examined multimodal interaction in an augmented reality environment, studying how users managed notifications while completing other tasks simultaneously [4]. They examined a large array of multimodal interaction methods combining gaze, pinch, point, speech, and touch. Their work found that each modality had its own advantages and disadvantages across various tasks, which is a key reason behind the motivation for allowing multiple interaction techniques rather than limiting a user to a single modality.

All in all, this collection of prior work establishes the potential and limitations of gesture-based HCI and the advantages of multimodal design.

3 Methodology

This project evaluates whether a small set of webcam-based hand gestures used alongside a standard mouse and keyboard can improve usability and task performance for common 3D object manipulation tasks on a laptop or desktop. The methodology covers three parts: the prototype system, the gesture set, and the user study design.

3.1 Prototype System

The prototype was built in Unity using C# and runs on a standard desktop or laptop with a webcam. Hand tracking is handled by the MediaPipe Unity plugin, which processes the live camera feed and outputs real-time hand landmark skeleton data from the webcam [8]. The hand detection is a two-part system: a BlazePalm detector first finds the hand in the frame, then a landmark model predicts 2.5D positions of the fingers and joints. The palm detector is very efficient and only runs on the first frame or when tracking is lost; otherwise, the hand region is based on the previous frame, keeping the system efficient enough for real-time desktop or laptop use.

Gesture recognition is built directly on top of these landmarks using geometric logic. For each frame, the state of all the fingers is tracked and determined to be either extended or curled based on the estimated y-position of each fingertip landmark compared against the corresponding knuckle landmark. Gestures are classified by particular finger combinations; each of the five defined hand gesture poses has its own unique combination of finger positions. To prevent accidental triggers, a gesture must remain stable for approximately 0.5 seconds before a command fires. All gesture events are time-stamped and logged automatically throughout testing sessions.

The 3D workspace presents participants with several simple objects—cubes, spheres, and cylinders—arranged in a neutral scene. A heads-up display along the top of the screen shows the name of the last recognized gesture or command and a webcam picture-in-picture. The webcam preview shows the landmark skeleton overlay on the participant's hand if visible, assisting participants in understanding what the system is detecting and reducing confusion during gesture input.

3.2 Gesture Set

There are five total gestures available to users during object manipulation tasks. Two of the gestures mirror common mouse behaviors for object control, and three substitute for keyboard shortcuts. The

Table 1: Gesture Set Summary

Gesture	Command	Keyboard Equiv.
Pinch	Select / Move	Mouse click + drag
Thumbs Up	Undo	Ctrl+Z
Closed Fist	Delete	Delete key
Index Point	Rotate	R key

set was designed to be visually distinct so that landmark-based detection would be more reliable in classifying each separate gesture. The gestures also aim to carry a natural physical association with their mapped actions, following the design principles defined by Wobbrock et al. [7].

The two mouse-like gestures are: (1) **Pinch** (thumb and index finger together) to select an object, with a sustained pinch and hand movement used to drag the selected object across the scene. This is intended as a direct alternative to the common mouse click-and-drag action.

The three shortcut-style gestures are: (2) **Thumbs Up** (all fingers curled, thumb extended upward) mapped to undo, (3) **Closed Fist** (all fingers fully curled) mapped to delete, and (4) **Index Point Up** (only index finger extended, thumb tucked) mapped to rotate. The thumbs-up-to-undo mapping was chosen for its simplicity and memorability. The fist-to-delete mapping was chosen to mimic a crushing or closing metaphor for removing content. Index-point-to-rotate was chosen because it is easily identifiable and signals focused attention on the object.

These gestures keep the hand in a natural, low-fatigue position for brief periods of time, consistent with the guidance from Hansberger et al. [3]. Each pose is also sufficiently distinct to make misclassification less likely: thumbs up and index point are distinguished by which landmark is highest, and the closed fist has no extended fingertips at all. Table 1 summarizes the complete gesture set.

3.3 User Study

The study uses a within-subjects design in which each participant completes the same task set under two conditions: keyboard and mouse only, and keyboard, mouse, and hand gestures. Condition order will be counterbalanced across participants to reduce order effects.

In both conditions, participants complete a series of structured object manipulation tasks: selecting a specified object, moving it to a target location, rotating it, undoing an action, and deleting an object. The task sequences are equivalent across conditions so that performance and usability can be compared directly. The dependent variables tracked are task completion time and the number of accidental or unintended inputs.

Following both sets of tasks, participants will also complete a short questionnaire asking which gestures felt most intuitive for their mapped commands, whether they preferred gesture or keyboard or mouse for each input type, and their overall satisfaction rating of each gesture on a 5-point scale. This combination of tracked performance data and post-session feedback allows the study to evaluate both usability and user preference.

References

[1] Farhad Farhadi-Niaki, S. Arvin Etemad, and Ali Arya. 2013. Design and Usability Evaluation of Gesture-Based Interaction with Desktop Applications. In *Human-Computer Interaction – INTERACT 2013*. Springer, 215–232. doi:10.1007/978-3-642-40480-1_14

[2] Liyuan Guo, Zhiyuan Lu, and Lin Yao. 2021. Human-Machine Interaction Sensing Technology Based on Hand Gesture Recognition: A Review. *IEEE Transactions on Human-Machine Systems* 51, 4 (2021), 300–309. doi:10.1109/THMS.2021.3086003

[3] Jeffrey T. Hansberger, Chao Peng, Shannon L. Mathis, Vaidyanath Areyur Shanthakumar, Sarah C. Meacham, Lizhou Cao, and Victoria R. Blakely. 2017. Dispelling the Gorilla Arm Syndrome: The Viability of Prolonged Gesture Interactions. In *Virtual, Augmented and Mixed Reality (VAMR 2017)*. Springer, 505–520. doi:10.1007/978-3-319-57987-0_41

[4] Daniel Plabst et al. 2025. Multimodal Interaction in Augmented Reality: Combining Gaze, Pinch, Point, Speech, and Touch for Notification Management. In *Proceedings of the CHI Conference on Human Factors in Computing Systems (CHI '25)*. ACM.

[5] Siddharth S. Rautaray and Anupam Agrawal. 2015. Vision Based Hand Gesture Recognition for Human Computer Interaction: A Survey. *Artificial Intelligence Review* 43, 1 (2015), 1–54. doi:10.1007/s10462-012-9356-9

[6] Richard Rodriguez, Brian T. Sullivan, Mayra Donaji Barrera Machuca, Anil Ufuk Batmaz, Cyane Tornatzky, and Francisco R. Ortega. 2024. An Artists' Perspectives on Natural Interactions for Virtual Reality 3D Sketching. In *Proceedings of the CHI Conference on Human Factors in Computing Systems (CHI '24)*. ACM, Article 633, 20 pages. doi:10.1145/3613904.3642758

[7] Jacob O. Wobbrock, Meredith Ringel Morris, and Andrew D. Wilson. 2009. User-defined Gestures for Surface Computing. In *Proceedings of the SIGCHI Conference on Human Factors in Computing Systems (CHI '09)*. ACM, 1083–1092. doi:10.1145/1518701.1518866

[8] Fan Zhang, Valentin Bazarevsky, Andrey Vakunov, Andrei Tkachenka, George Sung, Chuo-Ling Chang, and Matthias Grundmann. 2020. MediaPipe Hands: On-device Real-time Hand Tracking. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition Workshops*. https://arxiv.org/abs/2006.10214